

Chapter 6

LyonParallel—A Tool for Relative Termination of DPO Graph Rewriting

LyonParallel is a prototype tool, written in OCaml, for the analysis of DPO graph rewriting systems. It implements the notions of DPO rewriting systems on finite directed, edge-labeled multigraphs introduced in Chapter 2, the type graph method presented by Endrullis and Overbeek [39] and its extension in Chapter 3, and it also implements the morphism counting method in Chapter 4 and its extension in Chapter 5. For DPO graph rewriting systems with monic matches, the type graph method presented by Endrullis and Overbeek [39] is more powerful than the versions presented by Zantema et al. and Bruggink et al. [84, 19, 17]; these versions are implemented in TORPACyc [83] and Grez [20]. At the time of writing and to the best of our knowledge, the method presented by Overbeek and Endrullis has no publicly available implementation.

Figure 6.1 illustrates the workflow of LyonParallel.

Using the REPL (Read-Eval-Print Loop)—an interactive environment that reads user commands, evaluates them, prints results, and waits for the next input—users can select a set of rules encoded in the tool, then choose a method to partition the rules of the system into two sets $\mathcal{A}$ and $\mathcal{B}$ such that $\mathcal{A}$ terminates relative to $\mathcal{B}$. If such a partition is found, the tool replaces the current rule set with $\mathcal{B}$ and a new iteration begins; otherwise, users may try a different method. When the rule set becomes empty, the termination of the original system is proved.

6.1 DPO graph rewriting systems

6.1.1 Labeled graphs

A labeled graph is encoded in the tool as a structure with five fields: a list of nodes, a list of edges, a mapping that associates to each edge a source node, a mapping that associates to each edge a target node, and a mapping that associates to each edge a label. The representation of a labeled graph is constructed

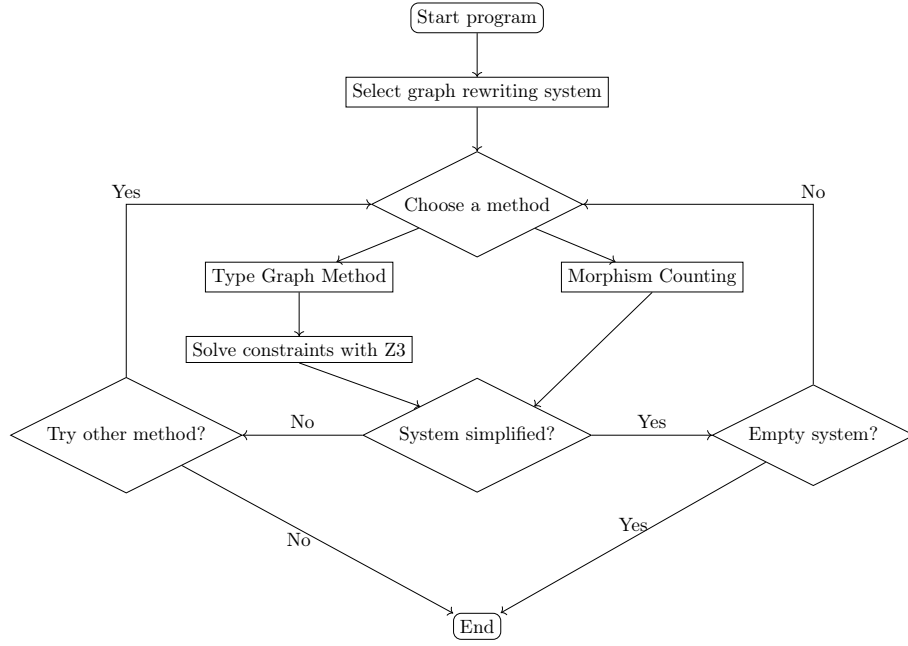
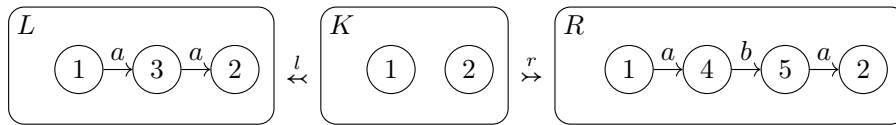


Figure 6.1: Workflow of LyonParallel

using the function `MGraph.fromList` with two parameters:

1. a list of integers representing the nodes of the graph, and
2. a list of four-tuples representing the labeled edges of the graph, where each tuple (s, lab, t, id) of type $int \times string \times int \times int$ represents an edge with source node s , label lab , target node t , and identifier id .

For example, consider the graph rewriting rule:



The representation of graphs K, L , and R in OCaml can be constructed by the following code:

```

let graph_K = MGraph.fromList [1;2] []
let graph_L = Mgraph.fromList [1;2;3]
  [(1, "a", 3, 1); (3, "a", 2, 2)]
let graph_R = Mgraph.fromList [1;2;4;5]
  [(1, "a", 4, 1); (4, "b", 5, 2); (5, "a", 2, 3)]

```

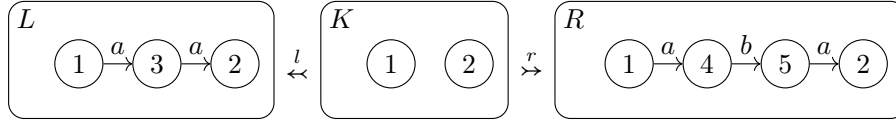
6.1.2 Homomorphisms

A homomorphism between two labeled graphs is encoded as a mapping from the nodes of the source graph to the nodes of the target graph, which preserves

the edge labels and the source and target nodes of each edge. Its representation can be constructed using the function `GraphHomomorphism.fromList` with four arguments:

1. the source graph,
2. the target graph,
3. a list of pairs, where each pair (s, t) means that node s of the source graph is mapped to node t of the target graph,
4. a list of pairs similar to the third argument, but for the edges.

For example, consider morphisms l and r of the rewriting rule:



Their representation in OCaml can be constructed using the following code, where `morphism_l` represents l and `morphism_r` represents r , respectively, and `graph_K`, `graph_L`, and `graph_R` represent graphs K , L , and R , respectively (see Section 6.1.1):

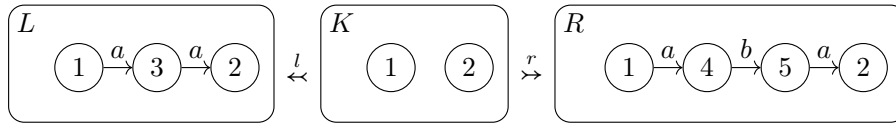
```
let morphism_l = GraphHomomorphism.fromList
                  graph_K graph_L [(1,1);(2,2)] []
let morphism_r = GraphHomomorphism.fromList
                  graph_K graph_R [(1,1);(2,2)] []
```

6.1.3 DPO rewriting rules and systems

A DPO rewriting rule is represented by a structure with two fields: a left-hand-side homomorphism and a right-hand side homomorphism. The representation of a rewriting rule can be constructed using the function

`GraphRewritingSystem.DPORule.fromHomos`

with the representations of the left- and right-hand side homomorphisms. For example, consider the rewriting rule:



The representation of the rewriting rule is constructed using the following OCaml expression, which creates the representation of a DPO rule from the representations of the left and right morphisms. Here, `morphism_l` and `morphism_r` denote the representations of l and r , respectively (see Section 6.1.2):

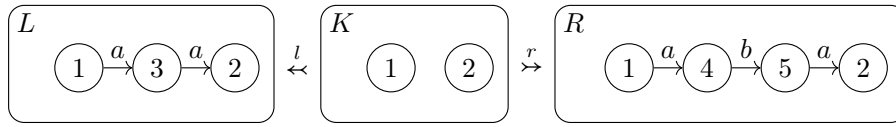
```
let rl = GraphRewritingSystem.DPORule.fromHomos
          morphism_l morphism_r
```

A DPO graph rewriting system is represented by a structure with three fields: the set of rules, the name of the system, and a Boolean indicating whether the system is restricted to monic matches. A representation of the DPO graph rewriting system can be constructed using the function

`GraphRewritingSystem.fromRulesListAndName`

with three arguments: a list of rewriting rules and the name of the system; the third parameter is `false` by default.

For example, consider the DPO graph rewriting system that consists of the rule:



It is represented by the following OCaml expression that creates a system from the singleton list containing the representation of the rule. Here, `rl` denotes the representation of the rewriting rule.

```

let bruggink14_ex_4_and_6 =
  GraphRewritingSystem.fromRulesListAndName
    [ rl ] "bruggink14_ex_4_and_6"

```

The function `GraphRewritingSystem.fromRulesListAndName` produces a system named `bruggink14_ex_4_and_6` from the singleton list `[rl]`.

6.1.4 User-defined rewriting systems

User-defined rewriting systems should be placed in the file

`lib/concretGraphRewritingSystems.ml`

To expose them, add each instance to the list

`available_graph_rewriting_systems`

in the same file.

For example, for the rewriting rule named `bruggink14_ex_4_and_6`, the following code should be added to the end of the file:

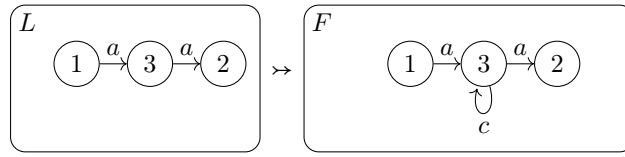
```

let available_graph_rewriting_systems =
  bruggink14_ex_4_and_6 ::
    available_graph_rewriting_systems

```

6.2 Ruler-graphs

A ruler-graph with one forbidden context as defined in Chapter 5 is represented by a structure with an underlying graph X and an injective graph homomorphism from X . Consider a ruler-graph (X, f) where X is the graph $\bigcirc \xrightarrow{a} \bigcirc \xrightarrow{a} \bigcirc$ and the injective graph homomorphism is shown below:



Its representation in OCaml can be constructed using the following code:

```

let graph_X = MGraph.fromList
  [1;2;3] [(1,"a",3,1);(3,"a",2,2)]
let ruler_graph_aa_notin_aca =
  let graph_F = MGraph.fromList
    [1;2;3] [(1,"a",3,1);(3,"c",3,3);(3,"a",
      2,2)] in
  let f = Homo.fromList graph_X graph_F
    [(1,1);(2,2);(3,3)] [(1,1);(2,2)] in
  {x: graph_X; fx = Some f}

```

Figure 6.2

Instances of ruler-graphs can be defined in the file `lib/ruler_graph.ml` of **LyonParallel**. To expose these instances to the REPL, they should be added to the list `ruler_graphs` in the same file. For example, for the instance of ruler-graph `ruler_graph_aa_notin_aca` shown in Figure 6.2, the following code should be added to the end of the file:

```

let ruler_graphs = ruler_graph_aa_notin_aca ::
  ruler_graphs

```

6.3 Installation and execution of LyonParallel

This section shows the installation process using **OPAM**—a package manager for OCaml. To install LyonParallel, one needs to have **OPAM** installed on one's system, then run the following command

```
>> opam switch create lyonParallel 5.2.1
```

and follow the instructions to create and switch to a new **OPAM** switch. The dependencies of LyonParallel are listed in the file `LyonParallel.opam` in the root directory of the project. To install these dependencies, run the following command:

```
>> opam install .
```

The project can be built and installed by running the following command:

```
>> dune build && dune install
```

The following command uninstalls the tool:

```
>> opam remove LyonParallel
```

Finally, the following command launches the interactive REPL of the tool:

```
>> LyonParallel
```

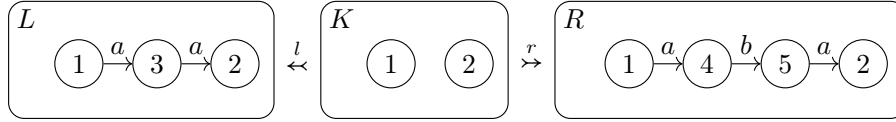
Upon launching, one should see:

```
>> Type "help" for a list of commands.
```

6.4 Relative termination analysis using LyonParallel

The LyonParallel REPL lets the user select a set of rewriting rules and run the tool's relative termination analysis techniques to test whether the set can be partitioned into two subsets $\mathcal{A}$ and $\mathcal{B}$ such that $\mathcal{A}$ terminates relative to $\mathcal{B}$. If such a partition is found, the REPL replaces the analysed rule set with $\mathcal{B}$ for further analysis; otherwise, the user can try alternative methods or different partitions.

The REPL command `systems` lists the available DPO graph rewriting systems; user-defined systems can be added as described in § 6.1. The predefined system consisting of the rule shown below is named `bruggink14_ex_4_and_6`:

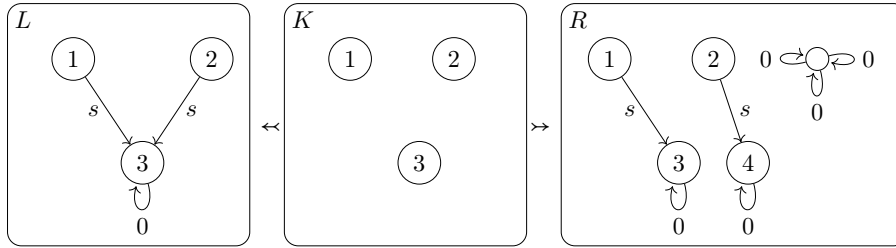


It appears in the list. Select it at the REPL by name:

```
>> select_system_by_name bruggink14_ex_4_and_6
```

6.4.1 Termination analysis using the morphism counting method

Consider the graph rewriting system presented in Example 4.26 with a single rule:



Its termination cannot be proved using the type graph method presented in [84, 19, 17, 39] and in Chapter 3. However, its relative termination with respect to the empty set can be proved using the morphism counting method.

The following sequence of commands selects the system, then uses the morphism counting method without forbidden patterns presented in Chapter 4 to analyse the relative termination of the system.

```
>> select_system_by_name
      plump18_ex6_rule_copy_variant
>> subgraph_counting_no_forbidden_context
```

The following command summarizes the result of the termination analysis.

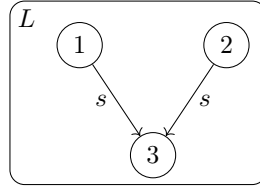
```
>> recap
```

The message displayed contains *Terminating: yes* when the method has proved the rule's relative termination with respect to the empty set; otherwise, the message contains *Terminating: unknown* and *Remaining Rules:*, followed by a list of rules to be analyzed.

Our implementation automatically tries every subgraph of the left-hand-side graphs of the system's rules as a candidate rule-graph until one succeeds. The chosen rule graph is then displayed in the analysis summary. For the system considered, the summary contains the information, shown below, which describes the employed rule graph.

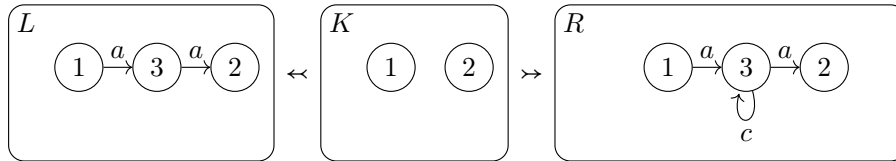
```
rule graph : nodes : [ 1;2;3 ]
arrows    : [ (1,s,3,1);(2,s,3,3) ]
```

The ruler-graph is illustrated below.



6.4.2 Termination analysis using morphism counting with antipatterns

Consider the graph rewriting system shown in Example 5.11, whose termination cannot be proven either by the morphism counting method without forbidden patterns introduced in Chapter 4 or by the subgraph counting method presented in [66]. Its unique rule is illustrated below.



The rule is predefined in the tool and can be selected with the REPL command:

```
>> select_system_by_name bruggink14_ex1
```

The REPL provides the command *rulergraphs* to list available ruler-graphs. User-defined ruler-graphs can be added to this list using the method presented in the previous section.

The ruler-graph shown in Figure 6.2 is predefined and can be selected with the REPL command:

```
>> select_ruler_graph aa_not_in_aca
```

To analyse the system's relative termination using the morphism counting method with a forbidden pattern (see Chapter 5) with the ruler-graph, run:

```
>> subgraph_counting_one_forbidden_context
```

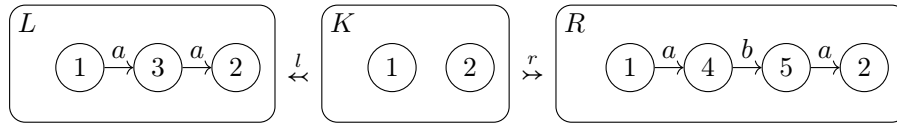
The following command displays a summary of the termination analysis.

```
>> recap
```

The message displayed contains *Terminating: yes*, which shows that the method applied successfully proved the relative termination of the rule with respect to the empty set of rules using the selected ruler-graph. If the method fails, the message contains *Terminating: unknown* and *Remaining Rules:*, followed by a list of rules to be analyzed.

6.4.3 Termination analysis using the type graph method

Consider the rewriting rule shown below:



The rewriting system consisting of the rule is predefined in the tool and can be selected with the REPL command:

```
>> select_system_by_name bruggink14_ex_4_and_6
```

For a selected system, the following command analyses relative termination of some non-empty subset of rules of the system with respect to the set of the remaining rules using the type graph method with the natural arithmetic semiring. The timeout is set to 30 seconds.

```
>> type_graph 30 n
```

This command is an instance of the following command pattern

```
>> type_graph [timeout] [semiring_1 semiring_2 ...]
```

This command pattern allows selecting additional semirings. In fact, there are 6 semirings available in the tool, which can be selected by adding flags in the set $\{a, t, n, A, T, N\}$ where A, T, N stand for the arctic, tropical, and arithmetic semirings over the (extended) natural numbers; a, t, n stand for the arctic, tropical, and arithmetic semirings over the (extended) real numbers. If multiple semirings are selected, the tool runs the type graph method in parallel for each chosen semiring. When some rules are removed the tool automatically starts a new iteration on the remaining rules using the same selected semirings.

For example, we can have the following command which launches the type graph method with the timeout set to 30 seconds and all 6 available semirings.

```
>> type_graph 30 a n t A N T
```


Type graph methods with different semirings will be launched in parallel and cooperate with each other. When a non-empty subset A of rules is proved to be terminating relative to the set of remaining rules B , a new iteration starts with the set of rules B .

The command `recap` displays a summary of the termination analysis. The summary contains `Terminating: yes` and `Remaining Rules: none`, showing that the method successfully proved the rule's relative termination with respect to the empty rule set. The summary also contains the information shown below.

```

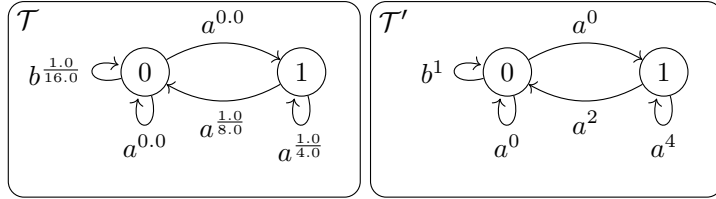
1 rule eliminated : rule 0
0 rule remaind   : None

0—a→0 : 0.0
1—a→1 : 1.0/4.0
0—a→1 : 0.0
0—b→0 : 1.0/16.0
1—a→0 : 1.0/8.0

0—a→0 : 0
1—a→1 : 4
0—a→1 : 0
0—b→0 : 1
1—a→0 : 2

```

The first block indicates that the rule whose representation is indexed by `0` (here the system's unique rule) can be eliminated using the type graph method over the real arctic semiring; no rules remain to be analyzed. The second block gives the information for constructing the type graph $\mathcal{T}$: two nodes and five edges with rational weights. The third block shows an equivalent type graph $\mathcal{T}'$ with integer weights obtained by multiplying the rational weights by their least common multiple. These type graphs are shown below, where the numbers inside the nodes are their identifiers.



6.5 Availability and license

LyonParallel is available at <https://github.com/Qi-tchi/LyonParallel/tree/thesis>. The project is distributed under the LGPL-2.1 License.